

SOLVING XOR PROBLEM USING MULTI LAYER PERCEPTRON

Submitted by N K U Sarada Anoushka | 1711086

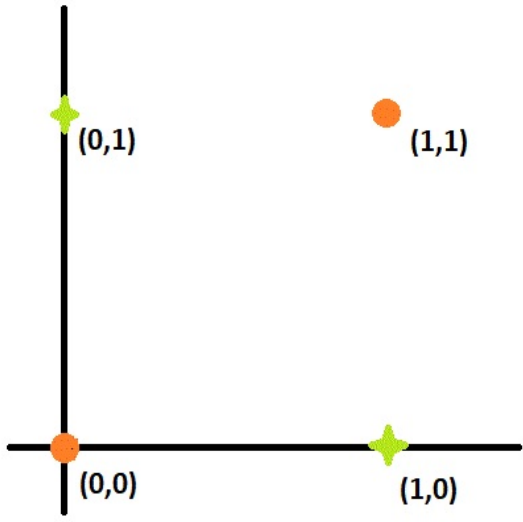
XOr logical Operator

XOr, or Exclusive Or, is a binary logical operator that takes in Boolean inputs and gives out True if and only if the two inputs are different. This logical operator is especially useful when we want to check two conditions that can't be simultaneously true. The following is the Truth table for XOr function

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

The XOr problem

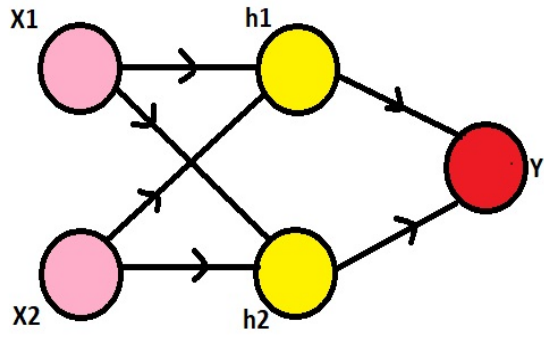
The XOr problem is that we need to build a Neural Network (a perceptron in our case) to produce the truth table related to the XOr logical operator. This is a binary classification problem. Hence, **supervised learning** is a better way to solve it. In this case, we will be using perceptrons. Uni layered perceptrons can only work with linearly separable data. But in the following diagram drawn in accordance with the truth table of the XOr loical operator, we can see that the data is **NOT** linearly separable.



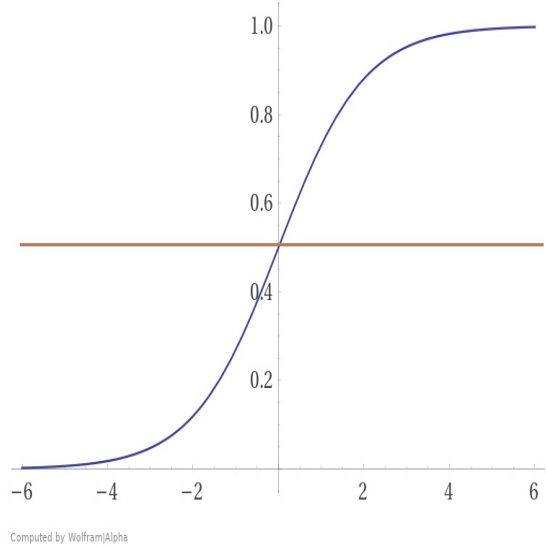
The Solution

To solve this problem, we add an extra layer to our vanilla perceptron, i.e., we create a **Multi Layered Perceptron** (or **MLP**). We call this extra layer as the **Hidden layer**. To build a perceptron, we first need to understand that the XOr gate can be written as a combination of AND gates, NOT gates and OR gates in the following way:
a XOr b = (a AND NOT b)OR(bAND NOTa)

The following is a plan for the perceptron.



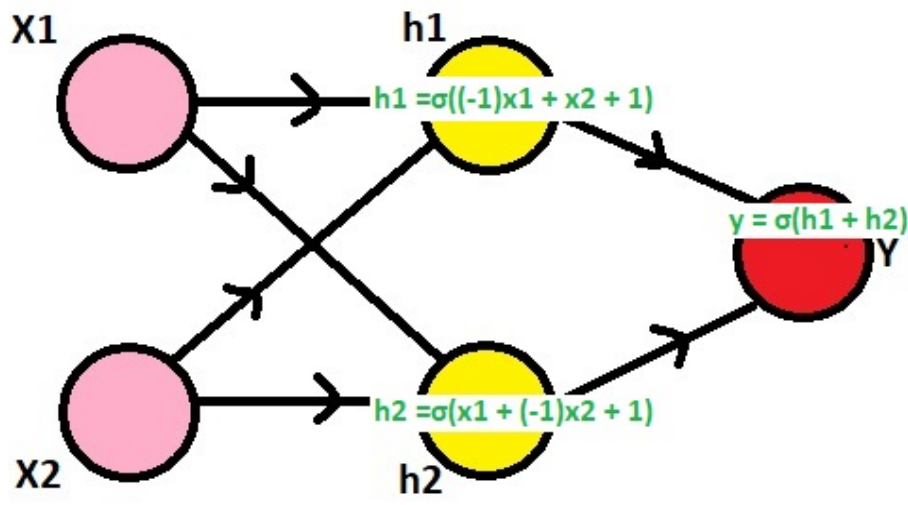
Here, we need to observe that our inputs are 0s and 1s. To make it a XOr gate, we will make the h1 node to perform the (x2 **AND NOT** x1) operation, the h2 node to perform (x1 **AND NOT** x2) operation and the y node to perform (h1 **OR** h2) operation. The NOT gate can be produced for an input a by writing (1-a), the AND gate can be produced for inputs a and b by writing (a.b) and the OR gate can be produced for inputs a and b by writing (a+b). Also, we'll use the sigmoid function as our activation function σ , i.e., $\sigma(x) = 1/(1+e^{(-x)})$ and the threshold for classification would be 0.5, i.e., any x with $\sigma(x)>0.5$ will be classified as 1 and others will be classified as 0.



Now, since we have all the information, we can go on to define h1, h2 and y. Using the formulae for AND, NOT and OR gates, we get:

1. $h1 = \sigma((1-x1) + x2) = \sigma((-1)x1 + x2 + 1)$
2. $h2 = \sigma(x1 + (1-x2)) = \sigma(x1 + (-1)x2 + 1)$
3. $y = \sigma(h1 + h2) = \sigma(h1 + h2 + 0)$

Hence, we have built a multi layered perceptron with the following weights and it predicts the output of a XOr logical operator.



References

- [1] Bradley, P. *The XOR Problem and Solution*. Website http://www.mind.ilstu.edu/curriculum/artificial_neural_net/xor_problem_and_solution.php
- [2] Ahire, J. B. (2017, Dec 17) *The XOR Problem in Neural Networks*. <https://medium.com/@jayeshbahire/the-xor-problem-in-neural-networks-50006411840b>
- [3] Victor Lavrenko. (2015, August 31). Neural Networks 6: solving XOR with a hidden layer [Video]. YouTube. <https://www.youtube.com/watch?v=kNPGXgzxoHw>
- [4] Sigmoid graph plotted using <https://www.wolframalpha.com/>